Husky HIL Main Instructions

FAST-LIO2 + SLAM Toolbox + Dijkstra Lattice

GNSS-denied simulation, hardware-in-the-loop and physical testing

Repository:	github.com/basu-repo/Journal_Work
Package:	`HIL/`
ROS distribution:	ROS 2 Jazzy
Prepared for:	Husky integration team
Document date:	6 November 2026

Important: Always begin disarmed. During computer-in-the-loop testing, keep the physical Husky motor interface disconnected or the E-stop engaged. During physical tests, keep an operator at the E-stop.

1 Choose the correct test mode

Mode	Sensors and vehicle	Command destination
Simulation	Gazebo sensors and simulated Husky	Gazebo `/cmd_vel`
Stationary HIL	Software runs on the physical Husky computer; Gazebo sensors and simulated Husky are used; physical Husky does not move	Isolated Gazebo command topic
Physical test	Physical LiDAR, physical IMU and physical Husky	Physical Husky command topic

2 Get the current package

```
git clone https://github.com/basu-repo/Journal_Work.git
cd Journal_Work/HIL

# For an existing checkout
git pull origin main
```

The main hardware package is `HIL/`. The `HIL/verified_stack/` directory reproduces the validated Bayland simulation.

3 Verified Bayland simulation

No hardware YAML changes are required for the packaged simulation.

```
cd HIL/verified_stack

python3 localization_experiment/run_diagnostic.py \
  --map bayland \
  --robot a300 \
  --model DijkstraLattice \
  --localization slam \
  --slam-backend slam_toolbox \
  --pose-backend fast_lio2 \
  --explore \
  --boundary-follow \
  --straight-only \
  --adaptive-horizon \
  --experimental-horizon-m 5 \
  --no-slam-scan-matching \
  --no-slam-loop-closing \
  --no-record-bag
```

Remove `-no-record-bag` when recording is required. The verified stack supplies the Bayland world, A300 model, simulated LiDAR and IMU, FAST-LIO2 configuration, SLAM Toolbox, Dijkstra Lattice and mission targets.

4 Gazebo topics used by the verified stack

Purpose	ROS 2 topic	Gazebo topic
Clock	`/clock`	`/clock`
3D LiDAR	`/lidar/points`	`/husky/lidar3d/points`
IMU	`/imu/data`	`/model/husky/imu`
Wheel odometry	`/odom`	`/wheel/odom`
Velocity command	`/cmd_vel`	`/cmd_vel`
GNSS, evaluation only	`/gps/fix`	`/husky/gps`

5 Stationary-Husky hardware-in-the-loop test

In this mode, the physical Husky computer runs localization and navigation. The physical Husky remains stationary. Gazebo supplies all sensor data, and only the simulated Husky moves.

Gazebo LiDAR + IMU → physical Husky computer
→ FAST-LIO2 + SLAM + Dijkstra Lattice
→ `/sim/husky/cmd_vel` → Gazebo Husky

Safety isolation

- Do not connect `/sim/husky/cmd_vel` to the Clearpath physical drive interface.
- Disable the physical motor driver or keep the physical E-stop engaged.
- Do not use physical LiDAR, IMU or wheel odometry in this test.
- Use the same `ROS_DOMAIN_ID` on the Gazebo and Husky computers.
- Verify bidirectional DDS communication before starting localization.

Isolated Gazebo command bridge

Change only the ROS side of the simulated command bridge:

```
- ros_topic_name: /sim/husky/cmd_vel
  gz_topic_name: /cmd_vel
  ros_type_name: geometry_msgs/msg/Twist
  gz_type_name: gz.msgs.Twist
  direction: ROS_TO_GZ
```

HIL robot profile

```
use_sim_time: true

localization:
  fastlio_config: fastlio.simulation.yaml
  pointcloud_topic: /lidar/points
  imu_topic: /imu/data
  calibration_confirmed: true

hardware_command_topic: /sim/husky/cmd_vel
```

Use the known simulated body-to-base calibration. Run navigation disarmed first and confirm the physical command interface receives no messages.

6 Prepare the physical robot configuration

For a physical test, Gazebo is removed from the control loop. Create site-specific files:

```
cd HIL
cp config/robot.example.yaml config/robot.site.yaml
cp config/fastlio.velodyne.example.yaml config/fastlio.site.yaml
cp config/mission.example.yaml config/mission.site.yaml
```

Values to change in robot.site.yaml

```
base_frame: base_link
map_frame: map
odom_frame: odom_lidar
odom_topic: /fastlio/odometry_base
use_sim_time: false

localization:
  calibration_confirmed: false
  fastlio_config: fastlio.site.yaml
  pointcloud_topic: <PHYSICAL_VELODYNE_TOPIC>
  imu_topic: <PHYSICAL_RAW_IMU_TOPIC>
  raw_odom_topic: /fastlio/odometry
  source_frame: camera_init
  source_body_frame: body
  body_to_base_translation: [x, y, z]
  body_to_base_quaternion: [x, y, z, w]

scan_topic: /scan
map_topic: /map
planner_command_topic: /hil/planner_cmd_vel
hardware_command_topic: <PHYSICAL_HUSKY_CMD_TOPIC>
```

Set `calibration_confirmed: true` only after verifying sensor clocks, frame directions and extrinsics.

Conservative first-test limits

```
max_linear_speed: 0.15
max_angular_speed: 0.20
linear_acceleration: 0.15
angular_acceleration: 0.25
reverse_speed: 0.08
```

Measure the physical footprint relative to `base_link`. Confirm that the physical command topic accepts `geometry_msgs/msg/Twist`; add a bridge if it expects `TwistStamped`.

7 Configure the physical Velodyne

The command below creates an editable site copy; it does not modify the example:

```
cp config/fastlio.velodyne.example.yaml config/fastlio.site.yaml
```

For a VLP-16 at 10 Hz with per-point time in seconds, begin with:

```
preprocess:
  lidar_type: 2
  scan_line: 16
  scan_rate: 10
  timestamp_unit: 0
  blind: 0.8
```

Timestamp code	Meaning
0	seconds
1	milliseconds
2	microseconds
3	nanoseconds

Enter the measured LiDAR-to-IMU transform:

```
mapping:
  extrinsic_T: [x, y, z]
  extrinsic_R: [1.0, 0.0, 0.0,
                0.0, 1.0, 0.0,
                0.0, 0.0, 1.0]
  extrinsic_est_en: false
```

The identity rotation is valid only when LiDAR and IMU axes point in exactly the same directions. Translation is in metres, with the LiDAR origin expressed in the IMU frame.

Required PointCloud2 fields

Each LiDAR point must normally contain `x`, `y`, `z`, `intensity`, `ring` and `time`. These are fields produced by the driver, not YAML values.

```
ros2 topic echo <velodyne-topic> --once --field fields
```

If the driver uses `t`, `timestamp`, or omits `ring` or `time`, stop and report the actual field description before running FAST-LIO2.

8 TF ownership

The required tree is:

```
map
 `-- odom_lidar
      `-- base_link
           |-- lidar_frame
           `-- imu_frame
```

Transform	Owner
`map -> odom_lidar`	SLAM Toolbox
`odom_lidar -> base_link`	supplied FAST-LIO adapter
`base_link -> lidar_frame`	robot URDF / robot_state_publisher
`base_link -> imu_frame`	robot URDF / robot_state_publisher

Do not disable `robot_state_publisher`. Disable or isolate only an existing localization component that publishes a competing dynamic parent for `base_link`.

```
ros2 topic info /tf --verbose
ros2 topic info /tf_static --verbose
ros2 run tf2_tools view_frames

ros2 topic echo <lidar-topic> --once --field header.frame_id
ros2 topic echo <imu-topic> --once --field header.frame_id

ros2 run tf2_ros tf2_echo base_link <lidar-frame>
ros2 run tf2_ros tf2_echo base_link <imu-frame>
```

There must be one publisher for each TF edge and one dynamic parent for `base_link`.

9 Mission coordinates

FAST-LIO2 initializes a local frame. The starting pose is normally near `(0,0)`; initial forward is positive X and left is positive Y.

```
targets:
  - {name: waypoint_1, x: 2.0, y: 0.0}
  - {name: goal, x: 4.0, y: 0.0}
```

Targets are metres in the active SLAM `map` frame, not latitude/longitude. Geographic targets require a surveyed start coordinate, initial bearing and conversion into local map X/Y before the mission.

10 Physical test sequence

Run from `HIL/` with the physical E-stop available.

Terminal 1: sensor preflight

```
source /opt/ros/jazzy/setup.bash
source fastlio_ws/install/setup.bash

python3 scripts/preflight.py \
  --robot config/robot.site.yaml \
  --sensors-only
```

This checks LiDAR and IMU receipt, rates, timestamps, frame IDs, required point fields and mounting TF.

Terminal 1: start localization

```
python3 scripts/start_localization.py \
  --robot config/robot.site.yaml
```

Keep the Husky stationary while FAST-LIO2 initializes.

Terminal 2: full preflight

```
python3 scripts/preflight.py \
  --robot config/robot.site.yaml
```

This additionally checks odometry, scan, occupancy map and the current `map -> base_link` transform.

Terminal 2: disarmed navigation

```
python3 run_hil.py \
  --robot config/robot.site.yaml \
  --mission config/mission.site.yaml
```

Disarmed mode must output zero physical velocity. Verify target placement, route, map alignment, TF freshness and watchdog behavior.

Terminal 2: armed controlled test

```
python3 run_hil.py \
  --robot config/robot.site.yaml \
  --mission config/mission.site.yaml \
  --arm
```

Arm only after all preceding checks pass.

11 Information the integration team must return

- `ros2 bag info <bag-directory>` output.
- Exact LiDAR and raw IMU topics and message types.
- LiDAR PointCloud2 fields, datatypes and per-point time unit.
- LiDAR and IMU rates and frame IDs.
- Physical command topic and message type.
- `/tf` and `/tf_static` publisher information and generated `frames.pdf`.
- Measured LiDAR-to-IMU and body-to-base calibration.
- Robot footprint and tested motion limits.
- Site configuration, repository commit and ROS bag for each trial.

12 Acceptance conditions

Before counting a trial as successful, verify:

- GNSS and ground truth are absent from localization and planning inputs.
- All required messages and TF transforms are fresh and use one clock.
- The disarmed watchdog publishes zero velocity.
- Command loss, stale sensors or loss of localization readiness stops the robot.
- The physical E-stop operates independently of navigation software.
- Waypoint and goal arrival are measured independently from the reported estimate.
- Contacts, localization error, mission time and final physical goal error are recorded.

Detailed supporting files are located under `HIL/docs/`.
Configuration values must be confirmed from the receiving robot, sensor driver and recorded data.